

Tutoriel — Mettre DatacenterMarket en production

Anavim Advisory · 10 juin 2026 · Phases 2 → 4 du plan de production
Prérequis : Phases 0 et 1 (sécurité) déjà déployées sur GitHub · Durée : 2-4 h sur 2-3 jours

Sommaire

- 1. La base de données MySQL cloud (~30 min)
- 2. L'hébergement gratuit sur Render (~45 min)
- 3. Stripe : test puis live (~45 min + vérification)
- 4. Monitoring et go-live (~30 min)
- 5. En cas de problème

Vue d'ensemble

Il reste trois briques externes à brancher, puis le déploiement :

Étape	Service	Compte à créer	Coût
1. Base de données	TiDB Serverless	tidbcloud.com	0 € (jusqu'à 5 Go)
2. Hébergement	Render (free tier)	render.com	0 €
2bis. Crons RGPD	GitHub Actions	déjà en place	0 €
3. Paiements	Stripe	stripe.com	0 € fixe (commission ~1,5 % + 0,25 € par vente)
4. Monitoring	Sentry + UptimeRobot	sentry.io, uptimerobot.com	0 € (tiers free)

Budget total : 0 € de coût fixe. Tu ne paies que la commission Stripe quand un client paie réellement.

⚠ **Limite du tier gratuit Render** : le serveur s'endort après 15 min sans trafic et met ~50 s à se réveiller. Le ping UptimeRobot toutes les 5 min (étape 4.2) le maintient éveillé en pratique. Quand les premiers clients arrivent, passe au plan Starter (~7 \$/mois).

Règle d'or : ne jamais mettre une clé secrète dans Git. Toutes les clés vont dans les variables d'environnement de l'hébergeur.

Étape 1 — La base de données MySQL cloud (~30 min)

Le projet utilise Drizzle ORM sur MySQL. Le plus simple et gratuit : **TiDB Serverless** (compatible MySQL).

1.1 Créer le cluster

1. Va sur **<https://tidbcloud.com>** → *Sign up* (compte Google possible).
2. Clique **Create Cluster** → choisis **Serverless** (gratuit jusqu'à 5 Go).
3. Région : **Frankfurt (eu-central-1)** — données en UE, cohérent avec le positionnement RGPD du site.
4. Une fois le cluster créé, clique **Connect** en haut à droite.
5. Choisis *Connect With* → **General**, génère le mot de passe et note la chaîne de connexion :

```
mysql://xxxxxx.root:MOT_DE_PASSE@gateway01.eu-central-1.prod.aws.tidbcloud.com:4000/test?ssl=
{"rejectUnauthorized":true}
```

1.2 Créer la base et pousser le schéma

Depuis ton PC, dans le dossier du projet :

```
cd C:\Users\Carlus\Projects\datacenter-market-v1

REM Mets la chaîne de connexion dans .env (remplace /test par /datacenter_market)
REM DATABASE_URL=mysql://xxxxxx.root:MOT_DE_PASSE@gateway01...:4000/datacenter_market?ssl=
{"rejectUnauthorized":true}

node_modules\.bin\pnpm db:push          REM crée les tables
node_modules\.bin\pnpm db:seed          REM insère le catalogue des 7 offres
node_modules\.bin\pnpm integration-check REM vérifie le tunnel complet
```

✅ **Critère de réussite** : `integration-check` affiche « Full funnel verified against the real database ».

💡 **Alternative OVH** : une instance « Web Cloud Databases MySQL » chez OVH fonctionne aussi (cohérent avec ton infra existante) — récupère l'URL de connexion au même format.

Étape 2 — L'hébergement gratuit sur Render (~45 min)

Le projet a un Dockerfile prêt. **Render** offre un tier gratuit qui suffit pour démarrer.

2.1 Créer le service Render

1. Va sur **`https://render.com`** → *Sign up with GitHub*.
2. **New** → **Web Service** → connecte le repo `datacenter-market-v1`.
3. Render détecte le Dockerfile automatiquement (Language : **Docker**).
4. **Instance Type : Free** ← c'est là que tu choisis le 0 €.
5. Région : **Frankfurt** (UE).

2.2 Configurer les variables d'environnement

Dans Render → ton service → onglet **Environment**, ajoute :

Variable	Valeur	Note
<code>DATABASE_URL</code>	la chaîne TiDB de l'étape 1	secrète
<code>JWT_SECRET</code>	64 caractères aléatoires	génère avec la commande ci-dessous
<code>OAUTH_SERVER_URL</code>	URL du serveur OAuth Manus	depuis ton espace Manus
<code>OWNER_OPEN_ID</code>	ton openId Manus	c'est ce qui fait de toi l'admin
<code>STRIPE_SECRET_KEY</code>	(étape 3)	<code>sk_test_</code> puis <code>sk_live_</code>
<code>STRIPE_WEBHOOK_SECRET</code>	(étape 3)	commence par <code>whsec_</code>
<code>SENTRY_DSN</code>	(étape 4)	optionnel au début

(Pas besoin de `PORT` : Render le fournit automatiquement et l'app le lit.)

Pour générer le `JWT_SECRET` :

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

⚠ **Important** : le serveur refuse de démarrer en production si `JWT_SECRET` fait moins de 32 caractères (protection ajoutée en Phase 1).

2.3 Les variables de build (VITE_*)

Ces variables sont **compilées dans le JavaScript public** au moment du build — n'y mets jamais un secret :

- `VITE_APP_ID` — identifiant de ton app Manus
- `VITE_OAUTH_PORTAL_URL` — URL du portail de login Manus
- `VITE_FRONTEND_FORGE_API_URL` / `VITE_FRONTEND_FORGE_API_KEY` — API Forge frontend (clé publique uniquement)

Ajoute-les aussi dans **Environment** : Render rend les variables disponibles pendant le build Docker. Si après déploiement le bouton de login pointe dans le vide, elles manquaient au build — ajoute-les puis **Manual Deploy** → **Clear build cache & deploy**.

2.4 Domaine et déploiement

1. Render te donne d'office un domaine gratuit en `xxx.onrender.com` — suffisant pour démarrer.
2. (Optionnel, toujours 0 € nouveau) : pour `compute.anavim-infra.com`, ajoute un **Custom Domain** dans Render → Settings puis crée le CNAME indiqué dans ta zone DNS OVH (le domaine `anavim-infra.com` est déjà payé). Certificat TLS automatique.
3. Chaque `git push` sur `main` redéploie automatiquement.

✅ **Critère de réussite** : `https://ton-domaine/health` répond `{"status":"ok"}` et la landing s'affiche.

2.5 Les crons RGPD — gratuits via GitHub Actions

Le tier gratuit Render n'inclut pas les cron jobs, mais **GitHub Actions** les exécute gratuitement. Le workflow `.github/workflows/crons.yml` est **déjà dans le repo** :

Cron	Commande	Fréquence
Purge des leads	<code>pnpm db:purge</code>	tous les jours à 03:00 UTC

Annulation commandes abandonnées

`pnpm db:cancel-stale`

toutes les heures (à :30)

Il te reste une seule chose à faire : GitHub → repo → **Settings** → **Secrets and variables** → **Actions** → ajoute le secret `DATABASE_URL` (la même chaîne TiDB). Teste ensuite avec **Actions** → **RGPD crons** → **Run workflow**.

Étape 3 — Stripe (~45 min + délai de vérification du compte)

3.1 Créer le compte

1. **<https://stripe.com>** → crée le compte au nom d'**Anavim Advisory SAS** (SIREN, IBAN, adresse 10 rue du Colisée).
2. La vérification d'identité peut prendre 1-2 jours — commence en **mode test** sans attendre.

3.2 Mode test d'abord (obligatoire)

1. Dashboard Stripe → active le **mode Test** (toggle en haut à droite).
2. **Développeurs** → **Clés API** : copie la clé secrète `sk_test_...` → variable `STRIPE_SECRET_KEY` dans Render.
3. **Développeurs** → **Webhooks** → **Ajouter un endpoint** :
 - URL : `https://ton-domaine/api/stripe/webhook`
 - Événement à écouter : `checkout.session.completed` (celui-là suffit)
4. Copie le **Signing secret** `whsec_...` → variable `STRIPE_WEBHOOK_SECRET`.
5. Redéploie (Render le fait automatiquement quand tu changes une variable).

3.3 Tester un paiement de bout en bout

1. Sur ton site : formulaire workload → choisis une offre → connecte-toi → **Confirm Order**.
2. Sur la page Stripe, paie avec la carte de test : **4242 4242 4242 4242**, date future, CVC 424.
3. Vérifie dans l'ordre :
 - redirection vers `/confirmation` avec la timeline de provisioning ;
 - Dashboard Stripe (mode test) → le paiement apparaît ;
 - ton dashboard admin (`/admin`) → le lead est passé à **converted** ;
 - Stripe → Webhooks → l'événement est en **Succeeded** (200).

⚠ **Point à valider explicitement** (hypothèse non testée du code) : choisis une offre **avec frais d'installation** (setup fee > 0) pour vérifier que Stripe accepte le line-item one-time dans une session subscription. Si Stripe renvoie une erreur à la création de session, le contournement est connu (facturer le setup via `subscription_data.add_invoice_items`).

3.4 Passage en live

1. Désactive le mode test → copie `sk_live_...` → remplace `STRIPE_SECRET_KEY`.
2. Recrée le webhook en mode live (nouveau `whsec_...`) → remplace `STRIPE_WEBHOOK_SECRET`.
3. Fais **un vrai paiement** avec ta propre carte sur l'offre la moins chère, puis rembourse-le depuis le dashboard Stripe.
4. Active **Stripe Tax** si tu factures de la TVA UE (B2B : autoliquidation avec numéro de TVA).

Étape 4 — Monitoring et go-live (~30 min)

4.1 Sentry (erreurs serveur)

1. <https://sentry.io> → compte gratuit → **Create Project** → plateforme **Node.js (Express)**.
2. Copie le **DSN** → variable `SENTRY_DSN` dans Render.
3. C'est tout — l'intégration est déjà câblée. Au prochain déploiement, le log affichera `[Sentry] Server monitoring initialized.`

4.2 Uptime

1. <https://uptimerobot.com> → compte gratuit → **Add Monitor** : HTTP(s), URL `https://ton-domaine/health`, intervalle 5 min.
2. Alerte e-mail vers `contact@anavim-infra.com`.

Double rôle sur le tier gratuit Render : le ping toutes les 5 min **empêche aussi le serveur de s'endormir** (veille après 15 min d'inactivité).

4.3 Décision télémétrie GPU

Le dashboard client affiche « Awaiting telemetry » car rien n'alimente les métriques. Trois options :

- (a) **Conseillé pour le lancement** : masquer le panneau (10 min de dev) ;
- (b) `pnpm db:telemetry --daemon` — données **simulées**, déconseillé face à de vrais clients ;
- (c) brancher une vraie ingestion plus tard (API des providers GPU).

4.4 Checklist finale avant d'annoncer le site

- ☐ `https://ton-domaine/health` → `{"status":"ok"}`
- ☐ Landing, formulaire, résultats, CGU/confidentialité/mentions légales s'affichent
- ☐ Login OAuth fonctionne et ton compte a bien le rôle admin (`/admin` accessible)

- ❑ Paiement test 4242... complet : lead `converted` + webhook 200
- ❑ Paiement réel + remboursement OK (mode live)
- ❑ Crons `db:purge` et `db:cancel-stale` planifiés
- ❑ Sentry reçoit les événements, UptimeRobot est vert
- ❑ Sauvegardes activées côté base (TiDB Serverless : automatiques)

En cas de problème

Symptôme	Cause probable	Solution
Le serveur ne démarre pas : « JWT_SECRET must be at least 32 characters »	Secret trop court	Régénère avec la commande de l'étape 2.2
« Database not available » sur le site	<code>DATABASE_URL</code> absente ou fausse	Vérifie la variable dans Render, teste avec <code>integration-check</code> en local
« Payments are not configured » au checkout	<code>STRIPE_SECRET_KEY</code> absente	Étape 3.2
Le paiement passe mais le lead reste « offered »	Webhook mal configuré	Vérifie l'URL du webhook et le <code>whsec_</code> , regarde Stripe → Webhooks → tentatives
Login OAuth en boucle ou erreur 400 « invalid state »	<code>VITE_OAUTH_PORTAL_URL</code> / <code>VITE_APP_ID</code> faux, ou domaine non déclaré côté Manus	Vérifie les variables de build et la config de l'app Manus
Pas de rôle admin après login	<code>OWNER_OPEN_ID</code> ne correspond pas	Récupère ton openId exact (table <code>users</code> après ta 1 ^{re} connexion) et corrige la variable
Première visite très lente (~50 s) puis tout va bien	Tier gratuit Render : le serveur dormait	Normal — le ping UptimeRobot le maintient éveillé ; plan Starter pour supprimer la veille

Document généré le 10 juin 2026 — accompagne `PRODUCTION-PLAN.md` (état technique) dans le dépôt `datacenter-market-v1` .

DatacenterMarket est un service d'Anavim Advisory SAS — 10 rue du Colisée, 75008 Paris.